

บทที่ 4

รูปแบบการโจมตี

รูปแบบของการโจมตีเครือข่ายคอมพิวเตอร์นั้นมีอยู่หลายประเภท โดยการโจมตีที่ผู้ไม่หวังดีนิยมใช้กันคือการโจมตีเพื่อให้ปิดบริการและการโจมตีอีกรูปแบบหนึ่งที่สร้างความเสียหายให้กับเซิร์ฟเวอร์ได้ค่อนข้างมากคือการโจมตีเว็บแอปพลิเคชัน โดยการโจมตีแต่ละประเภทยังมีดังต่อไปนี้

4.1 การโจมตีเพื่อให้ปิดบริการ (Denial of Services : DoS)

4.1.1 ความหมายของการโจมตีเพื่อให้ปิดบริการ

การโจมตีเพื่อให้ปิดบริการ หมายถึง การกระทำใดๆ ที่ทำให้ระบบเป้าหมายไม่สามารถให้บริการบางอย่างได้ หรือไม่ สามารถให้บริการต่อไปได้อีก โดยทั่วไปโจมตีที่ พอร์ตของทีซีพี/ไอพี ซึ่งเชื่อมต่อกับบริการ (Services) ที่รองรับพอร์ตนั้นๆ ดังนั้นการโจมตีพอร์ตจึงเท่ากับการโจมตีบริการของระบบนั่นเอง และอาจมีผลทำให้ระบบนั้นไม่สามารถให้บริการบางอย่างได้ หรือไม่ สามารถให้บริการใดๆ ได้เลย

4.1.2 ประเภทของการโจมตีเพื่อให้ปิดบริการ

ในที่นี้ประเภทของการโจมตีสามารถแบ่งได้ดังต่อไปนี้

4.1.2.1 การโจมตีประเภทอยู่ในชั้นทรานสปอร์ต หรือชั้นอินเทอร์เนต

การโจมตีในระดับชั้นนี้สามารถแบ่งได้เป็น 3 แบบหลักๆ ได้แก่

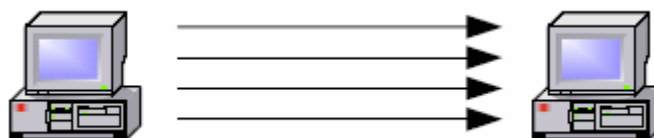
4.1.2.1.1 การส่งแพ็กเก็ตจำนวนมาก (Amount of Packets Sending)

การโจมตีแบบนี้เป็นการส่งแพ็กเก็ต ๓ ปริมาณมากเข้าไปยังระบบเป้าหมายอาจทำให้ระบบเป้าหมายไม่สามารถให้บริการบางอย่าง หรือไม่ สามารถทำงานต่อไปได้ ซึ่งแพ็กเก็ตที่ส่งออกไปนี้สามารถแบ่งออกได้เป็น

(1) แพ็กเก็ตข้อมูล (Data Packets)

การโจมตีวิธีนี้ทำได้โดยการส่งแพ็กเก็ตข้อมูลปริมาณมาก เมื่อข้อมูลเข้ามาสู่เครื่องเป้าหมายก็เก็บไว้ในบัฟเฟอร์ก่อนนำมาประมวลผลอีกครั้ง ดังนั้นหากส่งแพ็กเก็ตเข้ามาเป็นปริมาณมาก อาจทำให้บัฟเฟอร์ของเครื่องเป้าหมายไม่เพียงพอที่จะสามารถรองรับแพ็กเก็ตเหล่านั้นได้ทั้งหมด ซึ่งอาจทำให้เครื่องเป้าหมายให้บริการได้ช้าลงหรือต้องหยุดการให้บริการไปเลย ตัวอย่างการโจมตีประเภทนี้เช่น Ping Flood Attack เป็นต้น

Ping Flood เป็นการโจมตีในยุคแรกๆ ของ DoS หลักการคือส่ง ICMP Echo Request (รูปแบบเดียวกับคำสั่ง Ping) ไปยังเป้าหมายมากๆ ในระยะเวลาติดต่อกัน ทำให้เป้าหมายต้องคอยตอบ ICMP Echo Reply ตลอดเวลาจนไม่สามารถให้บริการอย่างอื่นได้ ความรุนแรงของการโจมตีขึ้นอยู่กับปริมาณแพ็กเก็ตที่โจมตีไปยังเครื่องเป้าหมาย หากเครื่องที่ทำการโจมตีมีประสิทธิภาพสูงและเครือข่ายมีแบนด์วิดท์มาก อาจส่งผลทำให้เครื่องเป้าหมายหยุดการทำงานลงได้



รูปที่ 4-1 การโจมตีด้วย Ping Flood Attack

ข้อสังเกตสำหรับการโจมตีประเภทนี้คือ จะปรากฏแพ็กเก็ต ICMP Echo Request และ ICMP Echo Reply ปริมาณมหาศาล โดยมีการรับส่งกันระหว่างเครื่องเป้าหมายที่ถูกโจมตีกับเครื่องอื่นๆ ที่อาจมีหรือไม่มีตัวตนในอินเทอร์เน็ตก็ได้ เนื่องจากกระบวนการสำคัญอย่างหนึ่งของการโจมตีลักษณะนี้คือ ผู้โจมตีต้องปลอมหมายเลขไอพี (IP Spoofing) เสมอ เพื่อป้องกันไม่ให้แพ็กเก็ต ICMP Echo Reply ถูกส่งกลับมายังเครื่องตัวเอง ซึ่งจะทำให้ผู้โจมตีได้รับผลกระทบจากการโจมตีด้วย และการปลอมไอพียังเป็นหลักประกันได้ว่าจะไม่สามารถติดตามได้ว่าผู้ใดเป็นผู้โจมตี รูปแบบแพ็กเก็ตที่เกิดขึ้นจากการโจมตีมีลักษณะดังนี้

```

14:49:43.217137 62.51.12.23 > 10.1.1.10 : icmp: echo request
14:49:43.217175 10.1.1.10 > 62.51.12.23 : icmp: echo reply
14:49:43.217195 62.51.12.23 > 10.1.1.10 : icmp: echo request
14:49:43.217219 10.1.1.10 > 62.51.12.23 : icmp: echo reply
14:49:43.217245 96.141.106.124 > 10.1.1.10 : icmp: echo request
14:49:43.217279 10.1.1.10 > 96.141.10.124 : icmp: echo reply
14:49:43.219017 172.19.251.18 > 10.1.1.10 : icmp: net 162.75.127.79 unreachable
14:49:43.237136 75.126.62.65 > 10.1.1.10 : icmp: echo request
14:49:43.237169 10.1.1.10 > 75.126.62.65 : icmp: echo reply
14:49:43.237193 75.126.62.65 > 10.1.1.10 : icmp: echo request
14:49:43.237216 10.1.1.10 > 75.126.62.65 : icmp: echo reply
14:49:43.237240 218.155.179.58 > 10.1.1.10 : icmp: echo request
14:49:43.237272 10.1.1.10 > 218.155.17.58 : icmp: echo reply

```

รูปที่ 4-2 รูปแบบแพ็กเก็ตที่เกิดขึ้นจากการโจมตีจาก Ping Flood Attack

นอกจาก Ping flooding Attack จะสร้างความเสียหายแก่เครื่องเป้าหมายแล้วยังสร้างความเสียหายแก่ระบบเครือข่ายของเครื่องเป้าหมายด้วย เพราะการโจมตีวิธีนี้จะสร้างแพ็กเก็ตเป็นจำนวนมากขึ้นในเครือข่ายที่เครื่องเป้าหมายตั้งอยู่ ทำให้ระบบเครือข่ายเกิดความคับคั่งของข้อมูล (Congestion) อาจส่งผลให้เครือข่ายเป็นอัมพาตได้ การป้องกันการโจมตีลักษณะนี้ทำได้โดยการกำหนดที่อุปกรณ์เราเตอร์หรือไฟร์วอลล์ โดย

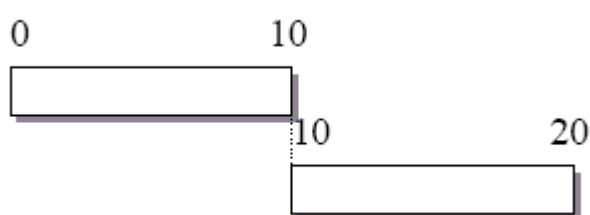
10:09:43.143 10.0.0.3 > isag14.ce.kmitl.ac.th.80: S 399259715:399259715(0)
 10:09:43.149 10.0.0.4 > isag14.ce.kmitl.ac.th.80: S 399259715:399259715(0)
 10:09:43.158 10.0.0.5 > isag14.ce.kmitl.ac.th.80: S 399259715:399259715(0)
 10:09:43.165 10.0.0.6 > isag14.ce.kmitl.ac.th.80: S 399259715:399259715(0)
 10:09:43.177 10.0.0.7 > isag14.ce.kmitl.ac.th.80: S 399259715:399259715(0)
 10:09:43.185 10.0.0.8 > isag14.ce.kmitl.ac.th.80: S 399259715:399259715(0)

รูปที่ 4-4 แพ็กเก็ตที่เกิดขึ้นจากการโจมตีแบบ SYN Flood

ในปัจจุบันนี้การโจมตีแบบ SYN Flood ถือได้ว่าเป็นการโจมตีที่ได้ผลและหาทางป้องกันได้ยาก เนื่องจากยากที่จะแยกลักษณะของแพ็กเก็ตที่ใช้ในการโจมตีกับแพ็กเก็ตที่ขอเริ่มต้นเชื่อมต่อทั่วไป นอกจากนี้ไฟร์วอลล์หรือเราเตอร์ทั่วไปยังไม่สามารถป้องกันการโจมตีประเภทนี้ได้อย่างสมบูรณ์ หนทางที่เป็นไปได้คือการใช้ระบบตรวจจับผู้บุกรุกทางระบบเครือข่ายทำการตรวจจับการโจมตี เพื่อนำข้อมูลจากการโจมตีกลับไปตั้งค่าอุปกรณ์เราเตอร์หรือไฟร์วอลล์เพื่อป้องกันการโจมตีมายังเซิร์ฟเวอร์

4.1.2.1.2 ความผิดปกติของแฟร็กเมนต์ (Abnormal Fragmentation)

การโจมตีวิธีนี้อาศัยหลักการแฟร็กเมนต์เซชันและรีแอสเซมเบิลที่กล่าวไว้ข้างต้น โดยทำให้แพ็กเก็ตนั้นต้องมีการรีแอสเซมเบิล (กำหนดค่า MF flag = 0) ซึ่งปกติการรีแอสเซมเบิลแพ็กเก็ตทั้งหมดต้องสามารถเชื่อมต่อกันได้สนิท ดังรูปที่ 4-5 แต่แพ็กเก็ตที่ผู้บุกรุกส่งไปมีการแก้ไขข้อมูลในบางฟิลด์ ทำให้เกิดความผิดปกติในกระบวนการรีแอสเซมเบิล ซึ่งการโจมตีในลักษณะนี้ แบ่งได้ดังต่อไปนี้

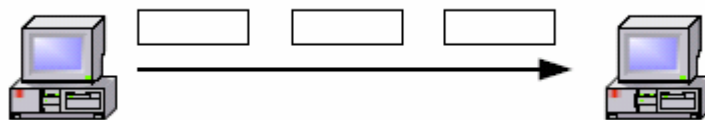


รูปที่ 4-5 แสดงการรีแอสเซมเบิลแบบปกติ

(1) การส่งแพ็กเก็ตที่มีลำดับผิดปกติ (Abnormal Sequences of Packets Sending)

ปกติการส่งแพ็กเก็ตมักเรียงตามลำดับกันไป หากไม่เรียงลำดับก็ต้องรอจนกว่าแพ็กเก็ตก่อนหน้านี้อมาถึง เพื่อเรียงลำดับแพ็กเก็ตที่เครื่องรับ แต่การโจมตีแบบนี้กลับส่งเฉพาะแพ็กเก็ตสุดท้าย เพื่อให้ระบบเป้าหมายรอแพ็กเก็ตก่อนหน้า และส่งไปเป็นปริมาณมากๆ ซึ่งจะส่งผลให้ระบบต้องจองทรัพยากร

ส่วนหนึ่งเพื่อรองรับแพ็กเก็ตที่ต้องรอเหล่านั้นในปริมาณมาก จนกระทั่งระบบไม่สามารถจัดหาทรัพยากรได้เพียงพอ ส่งผลให้ระบบไม่สามารถให้บริการได้อย่างอื่นได้



รูปที่ 4-6 แสดงการส่งเฉพาะแพ็กเก็ตสุดท้ายไปยังเป้าหมาย

โดยปกติแล้วการโจมตีในรูปแบบนี้ผู้โจมตีจะแก้ไขข้อมูลในฟิลด์แสดงลำดับของแพ็กเก็ต (Fragment Offset) ของแพ็กเก็ตไอพี ซึ่งเป็นส่วนที่แสดงลำดับของข้อมูลหลังจากกระบวนการแฟร็กเมนต์เดชัน โดยแก้ไขให้ส่งแพ็กเก็ตสุดท้ายหรือแพ็กเก็ตหลายๆ เพียงแพ็กเก็ตเดียวเลย ทำให้ระบบเป้าหมายต้องรอแพ็กเก็ตก่อนหน้านี้

(2) การส่งแพ็กเก็ตที่มีขนาดเหลื่อมกัน (Overlapped Packets' Size Sending)

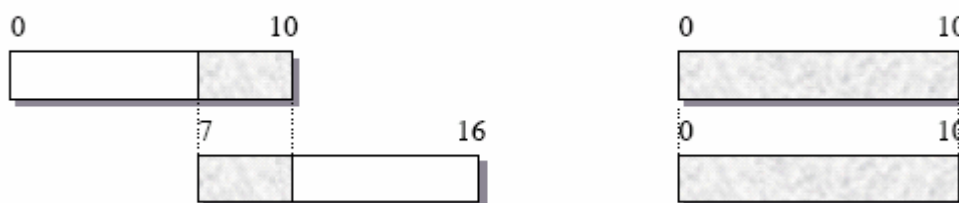
ปกติแพ็กเก็ตที่ส่งมาต้องนำมาต่อกันที่ระบบเป้าหมายได้พอดี แต่การโจมตีแบบนี้เป็นการส่งแพ็กเก็ตที่มีขนาดเหลื่อมกัน หรือซ้อนทับกันทำให้ข้อมูลเมื่อมาต่อกันแล้วเกิดความผิดพลาด หรือไม่

สามารถเชื่อมต่อกันได้โดยปกติแล้วการโจมตีแบบนี้ ผู้บุกรุกสามารถแก้ไขข้อมูลได้ 2 แห่งใหญ่ๆ ได้แก่

- การแก้ไขข้อมูลฟิลด์แสดงลำดับของแพ็กเก็ต (Fragment Offset) ของแพ็กเก็ตไอพี

หลังจากกระบวนการรีแอสเซมเบิล ซึ่งทำให้ลำดับในการส่งมีความผิดพลาด และอาจเกิดการเหลื่อมล้ำของแพ็กเก็ต กระบวนการรีแอสเซมเบิลอาจเกิดปัญหาได้

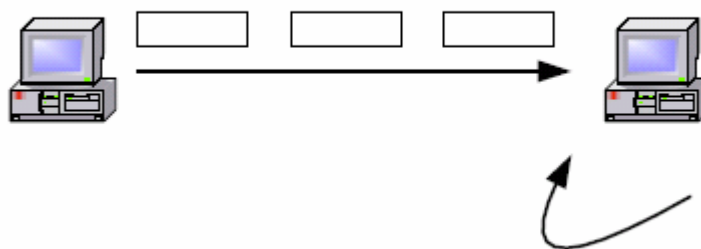
- การแก้ไขฟิลด์แสดงความยาวของ (Total Length) ของแพ็กเก็ตไอพี หลังจากกระบวนการรีแอสเซมเบิล ขนาดของแพ็กเก็ตที่มาต่อไม่พอดีกันทำให้ไม่สามารถรวมแพ็กเก็ตได้ หรือหากรวมได้ ข้อมูลที่ได้ก็ไม่ถูกต้อง



รูปที่ 4-7 แสดงการรีแอสเซมเบิลแบบแพ็กเก็ตมีขนาดเหลื่อมกัน

4.2.1.3 การส่งแพ็กเก็ตแบบวนลูป (Looping)

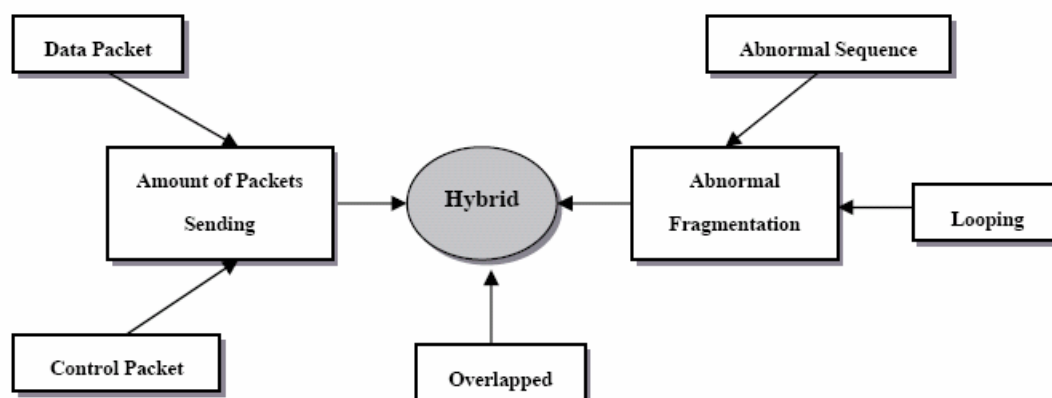
เป็นการโจมตีโดยการส่ง โดยกำหนดค่าแอดเดรสต้นทาง (Source Address) และแอดเดรสปลายทาง (Destination Address) ให้เหมือนกันทำให้เกิดการรับส่งวนไปวนมาอยู่ที่เครื่องเป้าหมายเอง เช่น LAND Attack ซึ่งเป็นโปรแกรมโจมตีที่มีการกำหนดแอดเดรสต้นทาง และแอดเดรสปลายทางเป็นค่าเดียวกัน คือ เป็นแอดเดรสของเครื่องเป้าหมายนั่นเอง ทำให้เกิดการส่งวนไปวนมาอยู่ที่เครื่องเป้าหมาย



รูปที่ 4-9 แสดงการโจมตีโดยส่งแพ็กเก็ตแบบวนลูป

4.2.1.4 การโจมตีแบบผสม (Hybrid)

คือ การโจมตีที่อาศัยวิธีการผสมกันระหว่างสามแบบแรกที่ได้กล่าวมาแล้ว ดังรูปที่ 4-10



รูปที่ 4-10 แสดงแผนภูมิแสดงประเภทของการโจมตีเพื่อให้บริการสำหรับสแตททีซีพี/ไอพี

4.2 การโจมตีเว็บแอปพลิเคชัน (Web Server Attacking)

เว็บแอปพลิเคชัน หมายถึงซอฟต์แวร์ที่สามารถเข้าใช้งานผ่านเว็บเบราว์เซอร์ องค์ประกอบของเว็บแอปพลิเคชันนั้นประกอบด้วย

1. เว็บเซิร์ฟเวอร์ (Web Server)
2. แอปพลิเคชันเซิร์ฟเวอร์ (Application Server)
3. ฐานข้อมูลเซิร์ฟเวอร์ (Database Server)

การโจมตีระบบเว็บแอปพลิเคชันนั้นสามารถโจมตีได้หลายอย่างโดยมีสาเหตุมาจาก

1. ความผิดพลาดของผู้ดูแลระบบที่ติดตั้งและตั้งค่าระบบต่างๆ ไม่ดีพอ
2. ความผิดพลาดจากผู้เขียนซอฟต์แวร์ที่เกี่ยวข้องกับการทำงานเช่น โปรแกรมไมโครซอฟต์ไอไอเอส (Microsoft IIS)
3. ความผิดพลาดจากผู้เขียนเว็บแอปพลิเคชันเองที่ไม่ได้ตระหนักถึงการทำงานให้เกิดความปลอดภัยในระบบ

ตัวอย่างเทคนิคต่างๆ ที่ใช้ในการโจมตีเว็บแอปพลิเคชัน

1. Hidden Field Manipulation
2. Cookie Poisoning
3. Backdoor and debug options
4. Application buffer overflows
5. Steath commanding
6. 3rd party misconfigurations
7. Known vulnerabilities
8. Parameter tempering
9. Cross site scripting
10. Forceful browsing

4.2.1 อิดคั่นฟิลด์ (Hidden Field)

สาเหตุเกิดจากเว็บแอปพลิเคชัน ส่งข้อมูลส่วนหนึ่งไปเก็บไว้ที่ไคลเอนท์โดยใช้ อิดคั่นฟิลด์แล้วนำค่าดังกล่าวมาใช้งานอีกครั้งหนึ่งตอนโพสเพื่อแสดงหน้าเพจถัดไป

ในการทำงานลักษณะนี้ผู้บุกรุกสามารถเปลี่ยนแปลงค่าอิดคั่นฟิลด์เพื่อสร้างความเสียหายเปลี่ยนแปลงการทำงานของเว็บแอปพลิเคชัน หรือให้ได้ผลลัพธ์ที่ต้องการ

4.2.2 คุกกี้อยุขฉันทิง (Cookie Poisoning)

ข้อมูลเกี่ยวกับเซสชันของการเชื่อมต่อไปยังเว็บแอปพลิเคชัน จะเก็บอยู่ในคุกกี (cookie) เมื่อมีการเปลี่ยนแปลงค่าในคุกกี้อยู่สามารถเปลี่ยนแปลงเซสชันไอดี (Session ID) ของการเชื่อมต่อได้

สำหรับข้อมูลที่เก็บอยู่ในคุกกี้นั้นอาจไม่มีความปลอดภัย คือไม่มีการเข้ารหัส หรือเข้ารหัสไว้ไม่ดีพอ

4.2.3 แบ็คดอร์และดีบั๊กออปชัน (Backdoor & Debug Options)

สำหรับแอปพลิเคชันที่พัฒนาขึ้นมาหลายๆแอปพลิเคชัน จะมีการดีบั๊ก (debug) การทำงานของแอปพลิเคชันโดยการป้อนพารามิเตอร์บางอย่างเข้าไปในระบบผ่านทางหน้าเว็บเพจ ซึ่งทำให้ผู้พัฒนาและผู้ดูแลระบบทราบค่าตัวแปรและการทำงานของแอปพลิเคชันนั้นๆได้ ทำให้ง่ายต่อการแก้ไขข้อผิดพลาด (bug) ต่างๆในระบบ โดยโค้ดส่วนดีบั๊กนี้ผู้พัฒนาเว็บแอปพลิเคชันจะเป็นคนใส่ในระบบเอง นอกจากนี้ผู้พัฒนาระบบอาจสร้างแบ็คดอร์ เพื่อใช้เป็นช่องทางในการติดต่อเข้าไปในระบบด้วย

การทำงานในลักษณะนี้จะเป็นอีกช่องทางหนึ่งที่ผู้ไม่หวังดีสามารถเข้าไปใช้งานระบบได้โดยได้สิทธิในทรัพยากรทุกอย่างในระบบ ซึ่งปกติจะได้สิทธิสูงสุดในระบบ ในการพัฒนาเว็บแอปพลิเคชันจึงควรปิดโหมดการดีบั๊ก (Disable debug mode) และไม่ควรมีแบ็คดอร์ในระบบด้วย

4.2.4 แอปพลิเคชันบัฟเฟอร์โอเวอร์โฟลว์ (Application Buffer Overflow)

การโจมตีเว็บแอปพลิเคชันในอีกรูปแบบหนึ่งคือการทำบัฟเฟอร์โอเวอร์โฟลว์ โดยจะทำการส่วนของเท็กซ์บ็อกซ์ (Text Box) ที่รับข้อมูลจากผู้ใช้งานเว็บเพจนั้นๆ การโจมตีทำได้โดยการป้อนอินพุตปริมาณมากๆลงในช่อง หรือส่วนในการรับอินพุตจากหน้าเว็บเพจ เมื่อเว็บเพจนั้นส่งข้อมูลไปยังเซิร์ฟเวอร์แล้ว ข้อมูลที่มีขนาดมากกว่าที่กำหนดไว้จะไปทำให้แอปพลิเคชันหยุดการทำงานได้ การป้องกันก็คือที่ฝั่งเซิร์ฟเวอร์ควรมีการตรวจสอบขนาดของข้อมูลที่ได้รับเข้ามาด้วย ไม่ให้เกินจากค่าที่กำหนดไว้

4.2.5 สเตลคอมมานด์ (Stealth Commanding)

เป็นการโจมตีผู้เว็บเซิร์ฟเวอร์โดยการส่งคำสั่งการทำงานต่างๆ แนบไปกับข้อมูลต่างๆในระบบ โดยการทำลักษณะนี้ได้นั้นเกิดจากการที่เว็บแอปพลิเคชันคิดว่าข้อมูลที่ได้รับมานั้นเป็นเพียงข้อมูลที่ไม่สามารถเอ็กซ์คิวต์ได้ ความเสียหายที่อาจจะเกิดขึ้นได้กับระบบก็คือ การถูกเปลี่ยนหน้าเว็บเพจ การปิดบริการ หรือการขโมยข้อมูลจากเซิร์ฟเวอร์ เป็นต้น

4.2.6 เทตปาร์ตี้มิสคอนฟิกูเรชัน (3rd Party Misconfiguration)

ความผิดพลาดอีกข้อหนึ่งที่ทำให้เกิดช่องโหว่ในระบบได้คือการตั้งค่าต่างๆ ในระบบไม่เหมาะสม หรือมีความผิดพลาดขณะติดตั้งโปรแกรม ซึ่งอาจทำให้เกิดปัญหาเช่น ยังมีการใช้รหัสผ่านที่โปรแกรมให้มา (Default Password) อยู่หรือค่าบางอย่างที่ทำให้เกิดความความปลอดภัยไม่ถูกเซ็ตไว้ ทำให้ผู้โจมตีระบบสามารถใช้ช่องโหว่นี้มาโจมตีระบบได้

4.2.7 โนลเวอร์เนอร์บิลิตี้ (Known Vulnerabilities)

ความไม่ปลอดภัยในลักษณะนี้เกิดจากจุดอ่อนในโปรแกรมที่เรานำมาใช้งาน ซึ่งโปรแกรมบางอย่างที่มีการใช้งานกันอย่างแพร่หลายก็อาจมีข้อผิดพลาดในโปรแกรมได้เช่นกัน เช่น ไมโครซอฟท์ ไอไอเอส ซึ่งปัญหาที่เกิดขึ้นอาจเกิดในจุดเล็กๆ ในระบบแต่ทำให้ระบบเกิดความไม่ปลอดภัยขึ้นได้

การแก้ปัญหานี้จะต้องแพตช์ (Patch) โปรแกรมที่มีปัญหา โดยการนำโปรแกรมสำหรับแก้ไขจุดอ่อนที่ออกโดยผู้พัฒนาแอปพลิเคชันนั้นๆ ซึ่งปกติแล้วจะมีการออกโปรแกรมแก้ไขออกมาอย่างรวดเร็ว แต่ปัญหาอยู่ที่ผู้ที่พัฒนาแอปพลิเคชันยกภาระการแก้ไขจุดอ่อนในระบบให้กับผู้ดูแลระบบเอง ซึ่งทำให้ช้ามากและมักจะไม่ทันการ

อันตรายจากปัญหาลักษณะนี้มักถูกถามอย่างรวดเร็ว โดยเฉพาะอย่างยิ่งเมื่อเกิดกับโปรแกรมที่มีการใช้งานกันอย่างแพร่หลายเช่น ไมโครซอฟท์ ไอไอเอส โดยสาเหตุที่ทำให้การถูกถามเป็นไปอย่างรวดเร็วนั้นจะเกิดจากผู้ดูแลระบบไม่ได้ติดตามข่าวจู่โจมของระบบที่ตัวเองดูแลอย่างสม่ำเสมอ แต่ผู้ไม่หวังดีกลับติดตามข่าวเหล่านี้อยู่เสมอๆ เมื่อมีข้อผิดพลาดของโปรแกรมเกิดขึ้น ก็จะมีผู้ประกาศตามหน้าเว็บไซต์ต่างๆทางด้านการปลอดภัยต่างๆ ซึ่งผู้ที่ทราบก่อนมักจะเป็นผู้ไม่หวังดีมากกว่าผู้ดูแลระบบ

4.2.8 พารามิเตอร์เทมเปอร์ริง (Parameter Tempering)

ปัญหานี้เกิดจากการที่เว็บแอปพลิเคชันใช้ค่าพารามิเตอร์จากไคลเอนต์ซึ่งการเปลี่ยนค่าพารามิเตอร์นั้นสามารถทำได้ง่ายดาย ซึ่งผู้เขียนเว็บแอปพลิเคชันนั้น มักจะคิดว่าค่าต่างๆจะเป็นค่าที่ถูกต้องแล้ว จะมีน้อยคนนักที่จะคำนึงถึงการเปลี่ยนแปลงพารามิเตอร์จะทำให้ระบบมีปัญหาอย่างไร

ปัญหาที่เกิดขึ้นจากการเปลี่ยนแปลงค่าพารามิเตอร์ไปเป็นค่าที่ไม่ถูกต้องนั้น ทำให้เกิดความไม่ปลอดภัยในข้อมูลหลายๆอย่าง เช่น การดึงข้อมูลของลูกค้าคนอื่นได้ การเปลี่ยนสิทธิของตนเองไปเป็นของคนอื่นๆเพื่อดึงข้อมูลส่วนตัวของคนอื่นๆ เป็นต้น

4.2.9 ครอสไซต์สคริปต์ (Cross Site Script)

ครอสไซต์สคริปต์ เป็นกระบวนการหนึ่งที่สำคัญจู่โจมของโฮสต์ที่ไม่มีการตรวจสอบว่าพารามิเตอร์ที่ป้อนเข้ามานั้นคือพารามิเตอร์จากไคลเอนต์จริงหรือไม่ จากจุดอ่อนดังกล่าวทำให้ผู้ไม่หวังดีสามารถโจมตีระบบได้โดยการฝากสคริปต์ไปรันที่เครื่องเป้าหมาย โดยผู้ใช้งานฝั่งไคลเอนต์เป็นผู้นำพาสคริปต์ไปยังเครื่องเป้าหมายได้

โดยการโจมตีจะมีการสร้างลิงค์ที่เว็บบอร์ดหรือหน้าเว็บเพจต่างๆ แล้วป้อนพารามิเตอร์เป็นจาวาสคริปต์ (javascript) รอให้คนอื่นๆติดต่อกับเครื่องไคลเอนต์ทำการคลิก แล้วสคริปต์จะทำงานทันที โดยความสามารถของสคริปต์นั้นจะมีตั้งแต่การขโมยเล็กน้อยๆ ไปจนถึงการขโมย session หรือข้อมูล Username และ Password ภายในเครื่องเป้าหมาย

4.2.10 ฟอร์ซฟูลบราวซิง (Forceful Browsing)

การโจมตีลักษณะนี้ผู้ไม่หวังดีจะใช้การคาดเดาว่าข้อมูลนั้นๆ อยู่ในไคลเอนต์ไหน แล้วป้อนตำแหน่งของข้อมูลนั้นๆ โดยตรงซึ่งปัญหานี้เกิดจากการใช้ดีฟอลต์ไฟล์ (Default File) ขณะติดตั้งโปรแกรม และไม่มีการลบไฟล์ที่ไม่ใช่ออกไปจากระบบ

ผลที่เกิดจากปัญหานี้คือข้อมูลต่างๆเช่น ล็อกไฟล์ โค้ดต้นแบบของโปรแกรมต่างๆ อาจถูกขโมยไปได้ถ้าเปิดสิทธิให้สามารถอ่านไฟล์หรือไคลเอนต์นั้นๆได้

และนอกจากการโจมตีเว็บแอปพลิเคชันทั้ง 10 รูปแบบที่ได้กล่าวมาแล้วยังมีการโจมตีเว็บแอปพลิเคชันในลักษณะประยุกต์ (Advanced Attack) ดังนี้

4.2.11 ซอร์สโค้ดดิสโคลด์เชอร์ (Source Code Disclosures)

การทำซอร์สโค้ดดิสโคลด์เชอร์ เป็นการให้จุดอ่อนในการออกแบบแอปพลิเคชัน ทำให้ผู้บุกรุกสามารถดึงข้อมูลของคอนฟิกูเรชันไฟล์ (Configuration file) หรือข้อมูลอื่นๆได้

ตัวอย่างของการทำซอร์สโค้ดคิสโค้ดเชอร์

- ข้อผิดพลาดใน WebLogic / WebSpere โดยข้อผิดพลาดนี้ผู้บุกรุกสามารถดึงข้อมูลของไฟล์นามสกุลเจเอสพี (JSP) และ เจเอชทีเอ็มแอล (JHTML) ได้ ซึ่งเกิดจากการตั้งค่าในเว็บเซิร์ฟเวอร์ผิดพลาด โดยการดึงข้อมูลนั้นสามารถทำได้โดยการเปลี่ยนตัวอักษร “jsp” ในยูอาร์แอล (URL) ให้กลายเป็นตัวพิมพ์ใหญ่ จะทำให้เซิร์ฟเวอร์ส่งรายละเอียดในไฟล์ .jsp มาแทนผลลัพธ์ในการทำงานของไฟล์ .jsp นั้น
- การทำซอร์สโค้ดคิสโค้ดเชอร์จะเป็นบั๊กในไมโครซอฟท์ไอโอเอส ซึ่งมีปัญหากับไฟล์ “.HTR” โดยผู้บุกรุกสามารถดูรายละเอียดในไฟล์นามสกุล .ASA และ .ASP ได้ ยกตัวอย่างยูอาร์แอลที่ทำให้เกิดปัญหาคือ
<http://10.0.0.1/global.asa+.httr>
 เมื่อเซิร์ฟเวอร์ได้รับการร้องขอยูอาร์แอลดังกล่าวแล้วจะทำงานโดย .httr จะทำให้ ISM.DLL ทำงานกับยูอาร์แอลดังกล่าวและเครื่องหมาย + จะทำให้ ISM.DLL ไม่ประมวลผล ตัวอักษรหลังเครื่องหมาย + นั้น
- ปัญหาของ Microsoft IIS showcode.asp ซึ่งเป็นโปรแกรมในการดูรายละเอียดในโค้ดของไฟล์ต่างๆได้ โดย showcode.asp จะถูกรวมอยู่กับไอโอเอสของ Windows NT Option Pack 4.0 โดยผู้บุกรุกที่ต้องการดูข้อมูลของไฟล์ต่างๆในระบบสามารถดูได้จากการป้อนยูอาร์แอล เช่น
<http://10.0.0.1/msadc/showcode.asp?Source=/msadc/../../../../../../path/to/file.name>

4.2.12 การโจมตีทางสถาปัตยกรรมของเว็บเซิร์ฟเวอร์ (Web Server Architecture Attack)

ในบางครั้งก็มีปัญหาในการออกแบบสถาปัตยกรรมของเว็บเซิร์ฟเวอร์ ทำให้เกิดช่องโหว่ขณะใช้งานเว็บเซิร์ฟเวอร์ได้ การโจมตีช่องโหว่ทางสถาปัตยกรรมของเว็บเซิร์ฟเวอร์นี้ จะให้วิธีการบายพาส (bypass) การทำงานบางส่วน of เว็บเซิร์ฟเวอร์ แล้วไปใช้งานบิวท์อินโพรซีเจอร์ (built-in procedure) ของเว็บเซิร์ฟเวอร์โดยตรง การแก้ปัญหานี้สามารถทำได้โดยการตรวจสอบสถาปัตยกรรมของเว็บเซิร์ฟเวอร์ให้ละเอียดเพื่อตรวจสอบดูว่ามีการทำงานที่นอกเหนือจากการทำงานปกติเกิดขึ้นในกรณีไหนบ้าง แล้วทำการแก้ไข

ในสถาปัตยกรรมเว็บเซิร์ฟเวอร์นั้นจะมีการตั้งค่าให้แฮนด์เลอร์ (handler) ต่างๆรับผิดชอบการประมวลผลไฟล์ต่างๆในระบบเมื่อถูกร้องขอเช่น เอกซ์ทีเอ็มแอลแฮนด์เลอร์ (html handler) จะทำงานเกี่ยวกับการรับส่งข้อมูลภายในไฟล์เอกซ์ทีเอ็มแอล ไปยังเครื่องที่ร้องขอ แต่ซีจีไอแฮนด์เลอร์ (cgi handler) จะรับผิดชอบในการเรียกให้ซีจีไอทำงานแต่ในบางกรณีจะมี default handler สำหรับการทำงานกับข้อมูลที่อยู่นอกเหนือหน้าที่ของแฮนด์เลอร์อื่นๆ ซึ่งอาจทำให้ผู้บุกรุกสามารถเรียกใช้ default handler นี้เพื่อทำการอ่านไฟล์ซีจีไอขึ้นมาแสดงผลได้ หรืออาจส่งค่าไฟล์เอกซ์ทีเอ็มแอลไปยังเจเอสพีแฮนด์เลอร์ (jsp handler) ทำให้ระบบคอมไพล์ไฟล์เอกซ์ทีเอ็มแอล โดยจาวาคอมไพเลอร์และเอ็กซีคิวต์โดยจาวารันไทม์ (java run-time) ในกรณีนี้ทำให้ผู้บุกรุกสามารถมีการทำงานบางอย่างในระบบได้ ดังตัวอย่างการทำแฮนด์เลอร์ฟอร์สซึ่ง

(handler forcing) ในเครื่องเว็บเซิร์ฟเวอร์ของซัน (Sun Java Web Server) โดยผู้บุกรุกสามารถป้อนยูอาร์แอลดังตัวอย่าง

```
http://10.0.0.2/servlet/com.sun.server.http.pagecompile.jsp.runtime.JspServlet/path/to/file.html
```

โดยจะมีการเรียกให้เซิร์ฟเลต (servlet) ทำงานจากการป้อนพาร์ท (path) /servlet/ แล้วเรียก PageCompile handler (Servlet) มาแฮนด์เคิล (handle) ไฟล์ข้อมูลข้างหลัง แล้วป้อนพาร์ทไปยังไฟล์ของข้อมูลที่ต้องการให้แฮนด์เคิลซึ่งในทางปฏิบัติผู้บุกรุกอาจจะป้อนข้อมูลที่เป็นโปรแกรมสำหรับการเชื่อมต่อทางไกลส่งไปให้จาวารันตามเป็นตัวเอ็กซีคิวต์ แล้วเปิดพอร์ตขึ้นรอรับการเชื่อมต่อจากผู้บุกรุกสำหรับการทำในลักษณะนี้จะทำให้การเชื่อมต่อนั้นมีสิทธิเทียบเท่าผู้ดูแลระบบทันที

4.2.13 เอสคิวแอลพอยซันนิงแอนด์อินเจกชัน (SQL Poisoning & Injections)

เป็นการโจมตีโดยใช้จุดอ่อนของการเขียนแอปพลิเคชัน ที่มีการใช้งานเอสคิวแอลสเตตเมนต์ (sql statement) โดยรับข้อมูลจากไคลเอนท์แต่ไม่ได้ตรวจสอบก่อนว่าข้อมูลที่รับเข้ามานั้นถูกต้องหรือไม่ซึ่งเอสคิวแอลสเตตเมนต์ จะเชื่อมต่อไปยังดีบีเอ็มเอสโดยตรง (ผ่าน SQL Query) ทำให้ผู้บุกรุกสามารถเพิ่มเติมและเปลี่ยนแปลงเอสคิวแอลสเตตเมนต์เพื่อให้ทำงานอื่นได้ ยกตัวอย่างโค้ดในการดึงข้อมูลฐานข้อมูล คือ

```
Dim sql_con, result, sql_qry
Const CONNECT_STRING =
"Provider=SQLOLEDB;SERVER=WEB_DB;UID=sa;PWD=xyzzy"
sql_qry= "SELECT * FROM PRODUCT WHERE ID ="
& Request.QueryString("DB")
```

```
Set objCon = Server.CreateObject("ADODB.Connection")
```

```
ObjCon.Open CONNECT_STRING
```

```
Set objRS = objCon.Execute(strSQL);
```

จากตัวอย่างโค้ดที่อยู่ในเว็บแอปพลิเคชันนั้น จะเห็นได้ว่าไม่มีการตรวจสอบค่าของอินพุทที่รับเข้ามาเลย ดังนั้นถ้ามีการร้องขอในลักษณะ

```
http://10.0.0.3/showtable.asp?ID=3+OR+1=1
```

ผลลัพธ์เมื่อโปรแกรมทำงานในคิวรี สเตตเมนต์ (Query statement) คือ

```
SELECT * FROM PRODUCT WHERE ID = 3 OR 1=1
```

ซึ่งจะผลให้มีการทำงานคำสั่งต่อไปนี้ที่แอปพลิเคชันเซิร์ฟเวอร์

```
SELECT * FROM PRODUCT WHERE ID = 3
```

```
DROP TABLE PRODUCT
```

นอกจากจะสามารถส่งคำสั่งเพื่อทำงานกับเอสคิวแอลสเตตเมนต์ได้แล้วยังสามารถส่งคำสั่งเพื่อการทำงานอื่นๆ ได้ด้วยเช่นกันยกตัวอย่าง

```
http://10.0.0.3/showtable.asp?ID=3%01EXEC+master..xp_cmdshell+'copy+\winnt\system32\cmd.exe+\inetpub\scripts'
```

ซึ่งจะส่งคำสั่งไปทำงานที่ฝั่งเซิร์ฟเวอร์คือ

Copy \winnt\system32\winnt\cmd.exe \inetpub\scripts

4.2.14 ไมโครซอฟท์ไอโอเอสยูนิโค้ดบั๊ก (Microsoft IIS Unicode bug)

สำหรับบั๊กที่สร้างความเสียหายต่อองค์กรธุรกิจอย่างมาก เห็นจะไม่พ้นยูนิโค้ดบั๊ก (Unicode bug) ในไมโครซอฟท์ไอโอเอส ซึ่งช่องโหว่นี้ทำให้ผู้บุกรุกสามารถส่งคำสั่งต่างๆ ไปทำงานยังเซิร์ฟเวอร์ได้อย่างง่ายดาย เพียงแค่ส่ง URL ที่มียูนิโค้ดที่มีปัญหาเข้าสู่ระบบ แล้วให้ระบบรับคำสั่งจาก URL ไปทำงานเช่น

<http://10.0.0.3/scripts/..%c0%af../winnt/system32/cmd.exe?c+dir>

ซึ่งจะส่งคำสั่ง dir ไปทำงานที่เว็บเซิร์ฟเวอร์ และส่งผลลัพธ์การทำงานมาที่หน้าจอบราวเซอร์